

## EXPLORER

v TP

> public

database.sql

server.js

package.json

.env.example

README.md

## PROJECT OVERVIEW

```
1 TP
2 |-- public/
3 |   |-- index.html
4 |   |-- style.css
5 |   |-- app.js
6 |-- database.sql
7 |-- server.js
8 |-- package.json
9 |-- .env.example
10 |-- README.md
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

database.sql

```
1 CREATE DATABASE IF NOT EXISTS train_reservation;
2 USE train_reservation;
3
4 CREATE TABLE passenger (
5     passenger_id BIGINT AUTO_INCREMENT PRIMARY KEY,
6     full_name VARCHAR(100) NOT NULL,
7     date_of_birth DATE NULL,
8     gender ENUM('MALE','FEMALE','OTHER') NULL,
9     phone VARCHAR(20) NOT NULL UNIQUE,
10    email VARCHAR(120) NULL UNIQUE,
11    id_type VARCHAR(30) NOT NULL,
12    id_number VARCHAR(50) NOT NULL UNIQUE,
13    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14 ) ENGINE=InnoDB;
15
16 CREATE TABLE train (
17     train_id BIGINT AUTO_INCREMENT PRIMARY KEY,
18     train_number VARCHAR(10) NOT NULL UNIQUE,
19     train_name VARCHAR(100) NOT NULL,
20     source_station VARCHAR(80) NOT NULL,
21     destination_station VARCHAR(80) NOT NULL,
22     distance_km DECIMAL(7,2) NOT NULL,
23     departure_time TIME NOT NULL,
24     arrival_time TIME NOT NULL,
25     active BOOLEAN NOT NULL DEFAULT TRUE
26 ) ENGINE=InnoDB;
27
28 CREATE TABLE train_service (
29     service_id BIGINT AUTO_INCREMENT PRIMARY KEY,
30     train_id BIGINT NOT NULL,
31     journey_date DATE NOT NULL,
32     service_status ENUM('SCHEDULED','CANCELLED','COMPLETED') NOT NULL DEFAULT 'SCHEDULED',
33     UNIQUE KEY uq_train_service (train_id, journey_date),
34     CONSTRAINT fk_service_train FOREIGN KEY (train_id) REFERENCES train(train_id)
35 ) ENGINE=InnoDB;
36
37 CREATE TABLE class_inventory (
38     inventory_id BIGINT AUTO_INCREMENT PRIMARY KEY,
39     service_id BIGINT NOT NULL,
40     travel_class ENUM('SL','3A','2A','1A','CC') NOT NULL,
41     total_seats SMALLINT NOT NULL,
42     available_seats SMALLINT NOT NULL,
43     base_rate_per_km DECIMAL(8,2) NOT NULL,
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

database.sql

continued - page 2

```
44     UNIQUE KEY uq_service_class (service_id, travel_class),
45     CONSTRAINT fk_inventory_service FOREIGN KEY (service_id) REFERENCES train_service(service_id)
46 ) ENGINE=InnoDB;
47
48 CREATE TABLE ticket (
49     ticket_id BIGINT AUTO_INCREMENT PRIMARY KEY,
50     pnr VARCHAR(12) NOT NULL UNIQUE,
51     passenger_id BIGINT NOT NULL,
52     service_id BIGINT NOT NULL,
53     travel_class ENUM('SL','3A','2A','1A','CC') NOT NULL,
54     booking_status ENUM('CONFIRMED','WAITLIST','CANCELLED') NOT NULL,
55     seat_number VARCHAR(10) NULL,
56     waitlist_number SMALLINT NULL,
57     fare DECIMAL(10,2) NOT NULL,
58     booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
59     cancelled_at TIMESTAMP NULL,
60     CONSTRAINT fk_ticket_passenger FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id),
61     CONSTRAINT fk_ticket_service FOREIGN KEY (service_id) REFERENCES train_service(service_id),
62     INDEX ix_waitlist (service_id, travel_class, booking_status, waitlist_number)
63 ) ENGINE=InnoDB;
64
65 DELIMITER $$
66
67 CREATE FUNCTION calculate_fare(p_service_id BIGINT, p_travel_class VARCHAR(10))
68 RETURNS DECIMAL(10,2) READS SQL DATA
69 BEGIN
70     DECLARE v_distance DECIMAL(7,2);
71     DECLARE v_rate DECIMAL(8,2);
72     SELECT tr.distance_km, ci.base_rate_per_km INTO v_distance, v_rate
73     FROM train_service ts
74     JOIN train tr ON tr.train_id = ts.train_id
75     JOIN class_inventory ci ON ci.service_id = ts.service_id
76     WHERE ts.service_id = p_service_id AND ci.travel_class = UPPER(p_travel_class);
77     RETURN ROUND((v_distance * v_rate + 25) * 1.05, 2);
78 END$$
79
80 CREATE TRIGGER ticket_inventory_after_insert
81 AFTER INSERT ON ticket FOR EACH ROW
82 BEGIN
83     IF NEW.booking_status = 'CONFIRMED' THEN
84         UPDATE class_inventory SET available_seats = available_seats - 1
85         WHERE service_id = NEW.service_id AND travel_class = NEW.travel_class;
86     END IF;
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

database.sql

```
87  END$$
88
89  CREATE TRIGGER ticket_inventory_after_update
90  AFTER UPDATE ON ticket FOR EACH ROW
91  BEGIN
92    IF OLD.booking_status <> 'CONFIRMED' AND NEW.booking_status = 'CONFIRMED' THEN
93      UPDATE class_inventory SET available_seats = available_seats - 1
94      WHERE service_id = NEW.service_id AND travel_class = NEW.travel_class;
95    ELSEIF OLD.booking_status = 'CONFIRMED' AND NEW.booking_status = 'CANCELLED' THEN
96      UPDATE class_inventory SET available_seats = available_seats + 1
97      WHERE service_id = NEW.service_id AND travel_class = NEW.travel_class;
98    END IF;
99  END$$
100
101  CREATE PROCEDURE promote_waitlist(IN p_service_id BIGINT, IN p_travel_class VARCHAR(10))
102  BEGIN
103    DECLARE v_available INT;
104    DECLARE v_ticket_id BIGINT;
105    DECLARE done INT DEFAULT 0;
106    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
107    SELECT available_seats INTO v_available FROM class_inventory
108    WHERE service_id = p_service_id AND travel_class = UPPER(p_travel_class) FOR UPDATE;
109    WHILE v_available > 0 AND done = 0 DO
110      SET v_ticket_id = NULL;
111      SELECT ticket_id INTO v_ticket_id FROM ticket
112      WHERE service_id = p_service_id AND travel_class = UPPER(p_travel_class)
113      AND booking_status = 'WAITLIST'
114      ORDER BY waitlist_number LIMIT 1 FOR UPDATE;
115      IF done = 0 THEN
116        UPDATE ticket SET booking_status = 'CONFIRMED',
117        seat_number = CONCAT(UPPER(p_travel_class), '-', v_available), waitlist_number = NULL
118        WHERE ticket_id = v_ticket_id;
119        SET v_available = v_available - 1;
120      END IF;
121    END WHILE;
122  END$$
123
124  CREATE PROCEDURE book_ticket(IN p_passenger_id BIGINT, IN p_service_id BIGINT, IN p_travel_class VARCHAR(10))
125  BEGIN
126    DECLARE v_available INT;
127    DECLARE v_status VARCHAR(15);
128    DECLARE v_waitlist INT;
129    DECLARE v_fare DECIMAL(10,2);
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

database.sql

continued - page 4

```
130 DECLARE v_pnr VARCHAR(12);
131 DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
132 START TRANSACTION;
133 SELECT service_status INTO v_status FROM train_service WHERE service_id = p_service_id FOR UPDATE;
134 IF v_status <> 'SCHEDULED' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Service is unavailable for booking'; END
IF;
135 SELECT available_seats INTO v_available FROM class_inventory
136 WHERE service_id = p_service_id AND travel_class = UPPER(p_travel_class) FOR UPDATE;
137 SET v_fare = calculate_fare(p_service_id, p_travel_class);
138 SET v_pnr = CONCAT('PNR', UPPER(SUBSTRING(REPLACE(UUID(), '-', ''), 1, 9)));
139 IF v_available > 0 THEN
140     INSERT INTO ticket (pnr, passenger_id, service_id, travel_class, booking_status, seat_number, fare)
141     VALUES (v_pnr, p_passenger_id, p_service_id, UPPER(p_travel_class), 'CONFIRMED', CONCAT(UPPER(p_travel_class),
'- ', v_available), v_fare);
142     SELECT v_pnr AS pnr, 'CONFIRMED' AS booking_status, v_fare AS fare;
143 ELSE
144     SELECT COALESCE(MAX(waitlist_number), 0) + 1 INTO v_waitlist FROM ticket
145     WHERE service_id = p_service_id AND travel_class = UPPER(p_travel_class);
146     INSERT INTO ticket (pnr, passenger_id, service_id, travel_class, booking_status, waitlist_number, fare)
147     VALUES (v_pnr, p_passenger_id, p_service_id, UPPER(p_travel_class), 'WAITLIST', v_waitlist, v_fare);
148     SELECT v_pnr AS pnr, 'WAITLIST' AS booking_status, v_fare AS fare;
149 END IF;
150 COMMIT;
151 END$$
152
153 CREATE PROCEDURE cancel_ticket(IN p_pnr VARCHAR(12))
154 BEGIN
155     DECLARE v_service_id BIGINT;
156     DECLARE v_class VARCHAR(10);
157     DECLARE v_status VARCHAR(12);
158     DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
159     START TRANSACTION;
160     SELECT service_id, travel_class, booking_status INTO v_service_id, v_class, v_status
161     FROM ticket WHERE pnr = UPPER(p_pnr) FOR UPDATE;
162     IF v_status = 'CANCELLED' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ticket is already cancelled'; END IF;
163     UPDATE ticket SET booking_status = 'CANCELLED', seat_number = NULL, waitlist_number = NULL,
164     cancelled_at = CURRENT_TIMESTAMP WHERE pnr = UPPER(p_pnr);
165     IF v_status = 'CONFIRMED' THEN CALL promote_waitlist(v_service_id, v_class); END IF;
166     COMMIT;
167 END$$
168 DELIMITER ;
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

## server.js

```
1  require('dotenv').config();
2  const path = require('path');
3  const express = require('express');
4  const mysql = require('mysql2/promise');
5
6  const app = express();
7  const pool = mysql.createPool({
8    host: process.env.DB_HOST || 'localhost', port: process.env.DB_PORT || 3306,
9    user: process.env.DB_USER || 'root', password: process.env.DB_PASSWORD || '',
10   database: process.env.DB_NAME || 'train_reservation', waitForConnections: true, connectionLimit: 10
11 });
12 app.use(express.json());
13 app.use(express.static(path.join(__dirname, 'public')));
14
15 app.get('/api/services', async (req, res, next) => {
16   try {
17     const [rows] = await pool.query(`SELECT ts.service_id, t.train_number, t.train_name, t.source_station,
18       t.destination_station, DATE_FORMAT(ts.journey_date, '%Y-%m-%d') AS journey_date,
19       TIME_FORMAT(t.departure_time, '%H:%i') AS departure_time, TIME_FORMAT(t.arrival_time, '%H:%i') AS arrival_time,
20       ci.travel_class, ci.available_seats, ci.total_seats
21     FROM train_service ts JOIN train t ON t.train_id = ts.train_id
22     JOIN class_inventory ci ON ci.service_id = ts.service_id
23     WHERE ts.service_status = 'SCHEDULED' ORDER BY ts.journey_date, t.train_number, ci.travel_class`);
24     res.json(rows);
25   } catch (error) { next(error); }
26 });
27
28 app.post('/api/passengers', async (req, res, next) => {
29   const { fullName, dateOfBirth, gender, phone, email, idType, idNumber } = req.body;
30   if (!fullName || !phone || !idType || !idNumber) return res.status(400).json({ message: 'Please complete all
required passenger fields.' });
31   try {
32     const [result] = await pool.execute(`INSERT INTO passenger
33       (full_name, date_of_birth, gender, phone, email, id_type, id_number) VALUES (?, ?, ?, ?, ?, ?, ?)`,
34     [fullName, dateOfBirth || null, gender || null, phone, email || null, idType, idNumber]);
35     res.status(201).json({ passengerId: result.insertId, message: 'Passenger registered successfully.' });
36   } catch (error) { next(error); }
37 });
38
39 app.post('/api/bookings', async (req, res, next) => {
40   const { passengerId, serviceId, travelClass } = req.body;
41   if (!passengerId || !serviceId || !travelClass) return res.status(400).json({ message: 'Passenger, service, and
class are required.' });
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

server.js

continued - page 2

```
42   try {
43     const [result] = await pool.query('CALL book_ticket(?, ?, ?)', [passengerId, serviceId, travelClass]);
44     res.status(201).json(result[0][0]);
45   } catch (error) { next(error); }
46 });
47
48 app.post('/api/cancellations', async (req, res, next) => {
49   if (!req.body.pnr) return res.status(400).json({ message: 'PNR is required.' });
50   try { await pool.query('CALL cancel_ticket(?)', [req.body.pnr]); res.json({ message: 'Ticket cancelled
successfully.' }); }
51   catch (error) { next(error); }
52 });
53
54 app.get('/api/tickets/:pnr', async (req, res, next) => {
55   try {
56     const [rows] = await pool.execute(`SELECT tk.pnr, tk.booking_status, tk.seat_number, tk.waitlist_number, tk.fare,
57       p.full_name, t.train_name, t.train_number, ts.journey_date, tk.travel_class
58       FROM ticket tk JOIN passenger p ON p.passenger_id = tk.passenger_id
59       JOIN train_service ts ON ts.service_id = tk.service_id JOIN train t ON t.train_id = ts.train_id WHERE tk.pnr =
?`, [req.params.pnr.toUpperCase()]);
60     if (!rows.length) return res.status(404).json({ message: 'Ticket not found.' });
61     res.json(rows[0]);
62   } catch (error) { next(error); }
63 });
64
65 app.use((error, req, res, next) => {
66   console.error(error);
67   res.status(400).json({ message: error.sqlMessage || error.message || 'Something went wrong.' });
68 });
69 app.listen(process.env.PORT || 3000, () => console.log('App running at http://localhost:' + (process.env.PORT ||
3000)));
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

public/index.html

```
1  <!doctype html>
2  <html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
  scale=1"><title>RailReserve</title><link rel="stylesheet" href="style.css"></head>
3  <body><header><a class="brand" href="#">RailReserve</a><nav><a href="#schedule">Schedules</a><a href="#book">Book
  ticket</a><a href="#manage">Manage ticket</a></nav></header>
4  <main><section class="hero"><p class="eyebrow">SMART TRAIN TRAVEL</p><h1>Your journey, booked simply.</h1><p>Find a
  service, reserve your seat, and manage your ticket in one place.</p><a class="button" href="#schedule">View
  schedules</a></section>
5  <section id="schedule"><div class="section-title"><div><p class="eyebrow">AVAILABLE SERVICES</p><h2>Train
  schedules</h2></div><button class="ghost" id="refreshServices">Refresh</button></div><div id="services"
  class="service-grid"><p>Loading schedules...</p></div></section>
6  <section id="book" class="two-column"><form id="passengerForm" class="card"><p class="eyebrow">STEP 1</p><h2>Register
  passenger</h2><div class="form-grid"><label>Full name*<input name="fullName" required></label><label>Phone*<input
  name="phone" required></label><label>Email<input name="email" type="email"></label><label>Date of birth<input
  name="dateOfBirth" type="date"></label><label>ID type*<select name="idType" required><option
  value="">Select</option><option>Aadhaar</option><option>Passport</option><option>Driving
  Licence</option></select></label><label>ID number*<input name="idNumber" required></label><label>Gender<select
  name="gender"><option value="">Prefer not to
  say</option><option>MALE</option><option>FEMALE</option><option>OTHER</option></select></label></div><button
  class="button">Register passenger</button><p class="hint" id="passengerResult">Your passenger ID will appear
  here.</p></form>
7  <form id="bookingForm" class="card"><p class="eyebrow">STEP 2</p><h2>Book ticket</h2><label>Passenger ID*<input
  name="passengerId" type="number" required></label><label>Service ID*<input name="serviceId" type="number"
  required></label><label>Travel class*<select name="travelClass" required><option value="">Select class</option><option
  >SL</option><option>3A</option><option>2A</option><option>1A</option><option>CC</option></select></label><button
  class="button">Confirm booking</button><p class="hint" id="bookingResult">Confirmed or waitlist status will appear
  here.</p></form></section>
8  <section id="manage" class="two-column"><form id="searchForm" class="card"><p class="eyebrow">TICKET
  DETAILS</p><h2>Find ticket</h2><label>PNR<input name="pnr" placeholder="PNRXXXXXXXX" required></label><button
  class="button">Search ticket</button><div class="result" id="ticketResult"></div></form><form id="cancelForm"
  class="card"><p class="eyebrow">CANCELLATION</p><h2>Cancel ticket</h2><label>PNR<input name="pnr"
  placeholder="PNRXXXXXXXX" required></label><button class="danger">Cancel ticket</button><p class="hint"
  id="cancelResult">Cancellation promotes the next waitlisted passenger automatically.</p></form></section></main>
9  <footer>RailReserve · Train ticket reservation system</footer><script src="app.js"></script></body></html>
```



EXPLORER

- v TP
  - > public
  - database.sql
  - server.js
  - package.json
  - .env.example
  - README.md

public/style.css

```
1 :root{--ink:#17212b;--muted:#66727d;--paper:#f7f6f2;--white:#fff;--orange:#e95d2a;--dark:#102a43;--line:#dfe3e6}{*box-sizing:border-box}html{scroll-behavior:smooth}body{margin:0;background:var(--paper);color:var(--ink);font:16px/1.5 Inter,Segoe UI,Arial,sans-serif}header{height:72px;background:var(--white);display:flex;align-items:center;justify-content:space-between;padding:0 8%;border-bottom:1px solid var(--line);position:sticky;top:0;z-index:2}.brand{font-size:1.4rem;font-weight:800;color:var(--dark);text-decoration:none}nav{display:flex;gap:22px}nav a{color:var(--muted);text-decoration:none;font-weight:600}main{max-width:1160px;margin:auto;padding:0 24px 80px}.hero{padding:110px 0 100px;background:linear-gradient(110deg,#102a43 0%,#1d4d68 65%,#e95d2a 160%);margin:0 calc(50% - 50vw) 70px;padding-left:max(8%,calc((100vw - 1112px)/2));color:white}.hero>{*max-width:610px}.hero h1{font-size:clamp(2.5rem,6vw,4.8rem);line-height:1.05;margin:10px 0 18px}.hero p:not(.eyebrow){font-size:1.15rem;opacity:.9}.eyebrow{font-size:.75rem;letter-spacing:.14em;font-weight:800;color:var(--orange);margin:0}.hero .eyebrow{color:#ffb199}.button,.danger,.ghost{border:0;border-radius:6px;padding:12px 18px;font-weight:700;cursor:pointer;font-size:.95rem}.button{background:var(--orange);color:white;text-decoration:none;display:inline-block;margin-top:20px}.ghost{background:transparent;border:1px solid var(--line);color:var(--dark)}.danger{background:#b92929;color:#fff;margin-top:20px}.section-title{display:flex;justify-content:space-between;align-items:end;margin-bottom:24px}h2{margin:4px 0 18px;font-size:1.6rem}.service-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(250px,1fr));gap:16px;margin-bottom:78px}.service,.card{background:var(--white);border:1px solid var(--line);border-radius:10px;padding:22px;box-shadow:0 4px 14px #102a4308}.service h3{margin:8px 0 4px}.route{color:var(--muted);font-size:.92rem}.tag{display:inline-block;background:#e6f3ed;color:#12633c;border-radius:99px;padding:4px 9px;font-size:.78rem;font-weight:800;margin-top:16px}.two-column{display:grid;grid-template-columns:1fr 1fr;gap:22px;margin-bottom:68px}.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:12px}label{display:block;font-weight:650;font-size:.88rem;margin:12px 0}input,select{display:block;width:100%;border:1px solid #cbd3d9;border-radius:5px;padding:11px;margin-top:5px;font:inherit;color:var(--ink);background:#fff}.hint,.result{font-size:.9rem;color:var(--muted);margin:18px 0 0}.result{white-space:pre-line;color:var(--ink)}.success{color:#147a48;font-weight:700}.error{color:#bb2222;font-weight:700}footer{text-align:center;padding:28px;background:var(--dark);color:#b7c8d8}@media(max-width:700px){header{padding:0 24px}nav{gap:12px;font-size:.85rem}.hero{padding:75px 24px;margin-left:-24px;margin-right:-24px}.two-column{grid-template-columns:1fr}.form-grid{grid-template-columns:1fr}}
```

## EXPLORER

v TP

&gt; public

database.sql

server.js

package.json

.env.example

README.md

public/app.js

```
1  const api = async (url, options = {}) => {
2    const response = await fetch(url, { headers: { 'Content-Type': 'application/json' }, ...options });
3    const data = await response.json();
4    if (!response.ok) throw new Error(data.message || 'Request failed.');
```

```
5    return data;
6  };
7  const setMessage = (id, message, type = '') => { const el = document.getElementById(id); el.textContent = message;
  el.className = `hint ${type}`; };
8
9  async function loadServices() {
10    const target = document.getElementById('services'); target.innerHTML = '<p>Loading schedules...</p>';
11    try {
12      const services = await api('/api/services');
```

```
13      target.innerHTML = services.length ? services.map(s => `<article class="service"><p class="eyebrow">SERVICE
  #${s.service_id}</p><h3>${s.train_name}</h3><strong>${s.train_number}</strong><p class="route">${s.source_station} →
  ${s.destination_station}</p><p>${s.journey_date} · ${s.departure_time} - ${s.arrival_time}</p><span
  class="tag">${s.travel_class} · ${s.available_seats}/${s.total_seats} seats</span></article>`).join('') : '<p>No
  scheduled services available.</p>';
14    } catch (error) { target.innerHTML = `<p class="error">${error.message}</p>`; }
15  }
16  document.getElementById('refreshServices').onclick = loadServices;
17  document.getElementById('passengerForm').onsubmit = async event => { event.preventDefault(); const data =
  Object.fromEntries(new FormData(event.target)); try { const result = await api('/api/passengers', { method:'POST',
  body:JSON.stringify(data) }); setMessage('passengerResult', `${result.message} Passenger ID: ${result.passengerId}`,
  'success'); event.target.reset(); } catch (e) { setMessage('passengerResult', e.message, 'error'); } };
18  document.getElementById('bookingForm').onsubmit = async event => { event.preventDefault(); const data =
  Object.fromEntries(new FormData(event.target)); try { const result = await api('/api/bookings', { method:'POST',
  body:JSON.stringify(data) }); setMessage('bookingResult', `PNR: ${result.pnr} · ${result.booking_status} · Fare:
  █${result.fare}`, 'success'); event.target.reset(); loadServices(); } catch (e) { setMessage('bookingResult',
  e.message, 'error'); } };
19  document.getElementById('searchForm').onsubmit = async event => { event.preventDefault(); const pnr = new
  FormData(event.target).get('pnr'); const target = document.getElementById('ticketResult'); try { const t = await
  api(`/api/tickets/${encodeURIComponent(pnr)}`); target.innerHTML = `<strong>${t.pnr} ·
  ${t.booking_status}</strong><br>${t.full_name}<br>${t.train_name} (${t.train_number})<br>${t.journey_date} ·
  ${t.travel_class}<br>Seat: ${t.seat_number || '-'} · Waitlist: ${t.waitlist_number || '-'}<br>Fare: █${t.fare}`; }
  catch (e) { target.textContent=e.message; target.className='result error'; } };
20  document.getElementById('cancelForm').onsubmit = async event => { event.preventDefault(); const pnr = new
  FormData(event.target).get('pnr'); try { const result = await api('/api/cancellations', { method:'POST',
  body:JSON.stringify({ pnr }) }); setMessage('cancelResult', result.message, 'success'); event.target.reset();
  loadServices(); } catch (e) { setMessage('cancelResult', e.message, 'error'); } };
21  loadServices();
```

EXPLORER

- v TP
  - > public
    - database.sql
    - server.js
    - package.json
    - .env.example
    - README.md

package.json

```
1 {
2   "name": "train-reservation-system",
3   "version": "1.0.0",
4   "private": true,
5   "scripts": { "start": "node server.js", "dev": "node --watch server.js" },
6   "dependencies": { "dotenv": "^16.4.7", "express": "^4.21.2", "mysql2": "^3.12.0" }
7 }
```

EXPLORER

v TP

> public

database.sql

server.js

package.json

.env.example

README.md

.env.example

```
1  PORT=3000
2  DB_HOST=localhost
3  DB_PORT=3306
4  DB_USER=root
5  DB_PASSWORD=your_mysql_password
6  DB_NAME=train_reservation
```

EXPLORER

v TP

- > public
- database.sql
- server.js
- package.json
- .env.example
- README.md

README.md

```
1  # RailReserve
2
3  ## Run
4
5  1. Open `database.sql` in MySQL Workbench and run it once.
6  2. Add your train, train-service, and class-inventory records in MySQL.
7  3. Copy `.env.example` to `.env` and enter your MySQL connection details.
8  4. Run `npm install`, then `npm start`.
9  5. Open `http://localhost:3000`.
10
11 The project has a Node.js/Express backend, MySQL database procedures and triggers, and a responsive
HTML/CSS/JavaScript frontend.
```